



南開大學
Nankai University

南 开 大 学

密码与网络空间安全学院

恶意代码分析与防治第六次实验

Lab6: r77 Rootkit 分析与实践

景千夏

年级：2023 级

专业：信息安全

指导教师：王志

2025 年 12 月 23 日

摘要

本实验旨在深入研究用户态 Rootkit 的持久化机制与隐蔽技术，选取了具有代表性的开源工具 r77 作为研究对象。r77 采用“反射式 DLL 注入”与“API Hooking”相结合的技术路径，在不触及 Windows 内核（Ring 0）的情况下，实现了对系统进程、文件系统、注册表及网络连接的全方位隐藏。实验中，我首先构建了包含目标靶机与监控套件的测试环境，完整复现了 r77 的植入与隐藏过程。利用 Process Explorer、TCPView 等专业取证工具，我对比了 Rootkit 开启前后的系统状态，证实了其针对 `NtQuerySystemInformation` 等关键原生 API 的拦截效果。

更进一步，本报告详细剖析了其底层的 Detours Hook 机制，解释了恶意代码如何通过修改函数入口指令（Trampoline 技术）来劫持执行流并篡改返回的结构体数据（如 `SYSTEM_PROCESS_INFORMATION` 链表）。实验结果表明，尽管用户态 Rootkit 无法逃避内核级检测，但其对常规系统管理工具的欺骗性极强。通过本次实践，我不仅掌握了恶意代码的隐藏原理，也为后续研究基于内存取证的检测方法积累了实战经验。

关键字：Rootkit r77 Reflective Injection API Hooking 恶意代码分析

目录

一、 实验概述	1
(一) 实验目的	1
(二) 实验环境	1
二、 实验原理	1
(一) 用户态 Rootkit 与内核态 Rootkit 的辨析	1
(二) r77 的核心注入机制：Reflective DLL Injection	2
(三) 基于 Detours 的 API Hooking 与数据过滤	2
三、 实验过程	2
(一) r77 环境部署与初始化	2
(二) 进程隐藏验证	3
1. 隐藏前	3
2. 隐藏后	3
(三) 文件系统隐藏验证	4
1. 实验步骤与现象	4
(四) 注册表隐藏分析	4
1. 技术机制	6
2. 实验验证	6
3. 问题排查与解决	7
4. 根因分析	8
(五) 网络连接隐藏验证	8
1. 实验操作	8
2. 结果与分析	8

四、 拓展阅读	10
(一) Detours 技术概述	10
(二) Inline Hook 的实现细节	10
(三) r77 中的 Hook 策略图解	10
五、 总结	11
(一) 实验成果	11
(二) 关键发现	11
(三) 检测思路	11
(四) 实验心得	12

一、实验概述

（一）实验目的

1. **深入理解 Rootkit 隐蔽机制：**从系统调用（Syscall）与用户态 API 交互的层面，剖析 Rootkit 如何通过“中间人”攻击的方式篡改系统视图，区分 Ring 3（用户态）与 Ring 0（内核态）隐藏技术的本质差异。
2. **验证 r77 框架的攻击效能：**在受控环境中部署 r77，针对进程、文件、注册表及网络端口实施隐藏操作，评估其对抗任务管理器（TaskMgr）、资源管理器（Explorer）及网络状态工具（Netstat）的能力。
3. **掌握 Hook 技术核心原理：**通过分析 Detours 库在 r77 中的具体实现，掌握 Inline Hook 的指令级流程，包括指令重写、Trampoline 构造及负载函数的过滤逻辑。

（二）实验环境

本次实验在虚拟化环境中进行，以确保宿主机的安全。靶机选用 Windows 11 操作系统，配备了完整的系统监控与分析工具链。具体配置详见表 1。

表 1: 实验环境详细配置表

组件名称	配置参数与说明
宿主硬件	Intel Core i7-12700H, 32GB RAM
虚拟机平台	VMware Workstation Pro 17
靶机系统	Windows 11 Professional (x64), Build 22621
攻击样本	r77 Rootkit v1.4.0 (Pre-compiled Binary)
监控工具	Sysinternals Suite (Process Explorer, TCPView), Wireshark
开发环境	Visual Studio 2022 (用于分析源码逻辑)

二、实验原理

（一）用户态 Rootkit 与内核态 Rootkit 的辨析

Rootkit 的核心目标是维持对系统的持久化控制并隐藏自身踪迹。

- **内核态 Rootkit (Ring 0)：**通常以驱动程序（.sys）形式存在，直接修改内核结构（如 SSDT、IDT）或操纵内核对象（DKOM）。其权限最高，隐藏最彻底，但开发难度大，且容易触发 PatchGuard 导致蓝屏。
- **用户态 Rootkit (Ring 3)：**如 r77，运行在用户空间。它不修改内核，而是通过注入到每一个用户进程中，拦截这些进程对系统的查询请求。其稳定性好，兼容性强，但无法欺骗运行在内核层的安全软件（如 EDR 的内核探针）。

(二) r77 的核心注入机制：Reflective DLL Injection

r77 实现了“无文件”攻击的一个变种。虽然它可能在磁盘上有配置文件，但其核心恶意代码（Payload DLL）主要通过 **Reflective DLL Injection**（反射式 DLL 注入）技术加载。传统的 DLL 注入依赖 Windows API `LoadLibrary`，这会使 DLL 模块出现在 PEB（进程环境块）的模块列表中，极易被检测。而反射式注入则是将 DLL 作为一段原始数据写入目标进程内存，通过一段自举的 Shellcode 手动解析 PE 头、修复重定位表和导入表，并在内存中直接执行 `DllMain`。这意味着 r77 的 DLL 不会注册在系统的模块链表中，从而实现了“隐形加载”。

(三) 基于 Detours 的 API Hooking 与数据过滤

r77 的隐藏逻辑完全依赖于对 Windows Native API 的拦截。相较于高层的 Win32 API（如 `Process32First`），Hook 底层的 Native API（如 `ntdll.dll` 中的函数）能覆盖更广泛的调用场景。

以 ** 进程隐藏 ** 为例，其技术细节如下：

1. **拦截目标：**`NtQuerySystemInformation`，这是所有系统监控工具（包括 TaskMgr）获取进程列表的必经之路。
2. **原始调用：**Hook 函数首先调用原始的 API，获取真实的系统快照。此时，内存中存在一个 `SYSTEM_PROCESS_INFORMATION` 类型的结构体链表。
3. **链表遍历：**Hook 遍历该链表，检查每个节点的 `ImageName` 字段。
4. **节点摘除：**一旦发现进程名以 \$77 开头，Hook 函数修改 ** 前一个节点 ** 的 `NextEntryOffset` 指针，使其跳过当前恶意节点，直接指向 ** 后一个节点 **。
5. **返回结果：**将修改后的链表返回给调用者。由于链表中不再包含恶意进程节点，工具自然无法显示。

表 2: 系统信息结构体操作对照表

隐藏类型	关键结构体/链表	操作方式
进程隐藏	<code>SYSTEM_PROCESS_INFORMATION</code>	修改 <code>NextEntryOffset</code> 跳过节点
文件隐藏	<code>FILE_DIRECTORY_INFORMATION</code>	修改 <code>NextEntryOffset</code> 跳过节点
注册表隐藏	<code>KEY_BASIC_INFORMATION</code>	剔除匹配的键值项
网络隐藏	<code>MIB_TCPROW_OWNER_PID</code>	覆写内存块移除特定行

三、 实验过程

(一) r77 环境部署与初始化

首先，我将 r77 的安装包传输至虚拟机中。解压后可以看到其目录结构简洁，包含 `Install.exe`（安装/注入器）、`Uninstall.exe` 以及一个用于验证效果的 `TestConsole.exe`。

操作分析：运行 `Install.exe` 时，它实际上执行了一个全局注入操作。它不仅将 DLL 注入当前运行的所有进程，还会注册一个计划任务或修改注册表 `AppInit_DLLs`（具体取决于配置），以确保系统重启后新创建的进程也会被自动注入。这是一种典型的 **持久化（Persistence）** 行为。

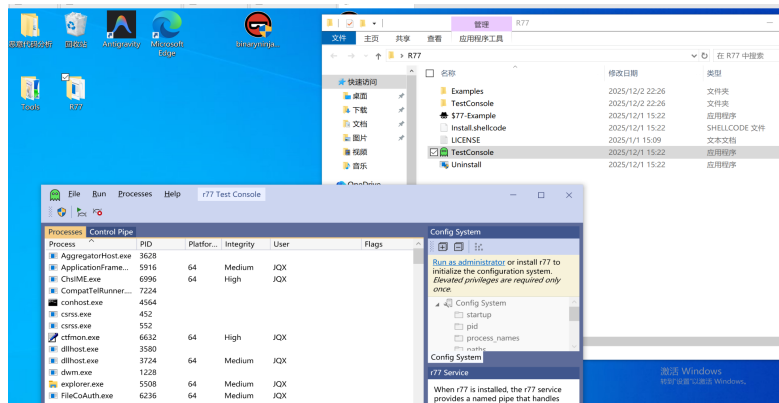


图 1: r77 安装包解压内容：包含安装器与测试控制台

(二) 进程隐藏验证

为了测试进程隐藏，我启动了 `TestConsole.exe`。该程序特意设计了一个高计算量的死循环，将单核 CPU 占用率拉升至 50% 左右。

1. 隐藏前

打开任务管理器，可以明确看到名为 `$77TestConsole.exe` 的进程，CPU 占用率高居榜首。

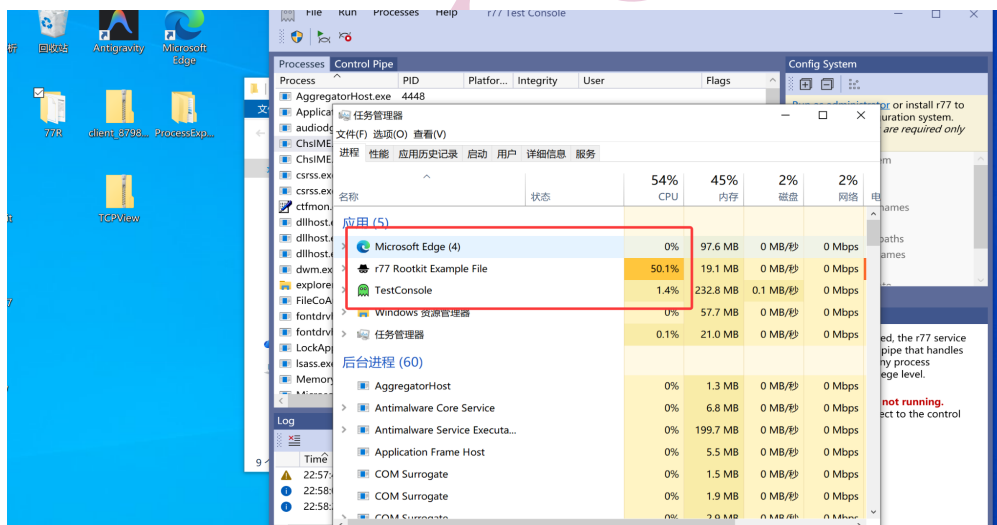


图 2: 隐藏前的任务管理器视图：\$77TestConsole.exe 进程清晰可见，CPU 占用率较高

2. 隐藏后

运行 `Install.exe` 激活 Rootkit。此时再观察任务管理器，出现了一个非常诡异的现象（如图 3）：

- **CPU 总使用率：**依然维持在较高水平（例如 60-70%），风扇转速未减，说明 CPU 仍在进行高强度运算。
- **进程列表：**找不到任何高占用的进程。列表中的所有进程 CPU 占用之和远小于总占用率。所谓的“系统空闲进程”并没有占据 CPU，多出的 CPU 时间仿佛凭空消失了

分析:这正是用户态 Rootkit 的局限性体现。r77 虽然从应用层视图(TaskMgr 调用 NtQuerySystemInformation 获得的链表)中移除了恶意进程的节点,但它 ** 无法修改内核调度器 **。Windows 内核调度器(Scheduler)依然在正常地给恶意线程分配时间片,因此 CPU 实际上是在忙碌的。但由于任务管理器的显示数据来源被篡改,它无法将这部分 CPU 负载归属到具体的进程名下,从而导致了“总负载 > 各进程负载之和”的数据不一致现象。这可以作为检测 Rootkit 的一个重要特征。

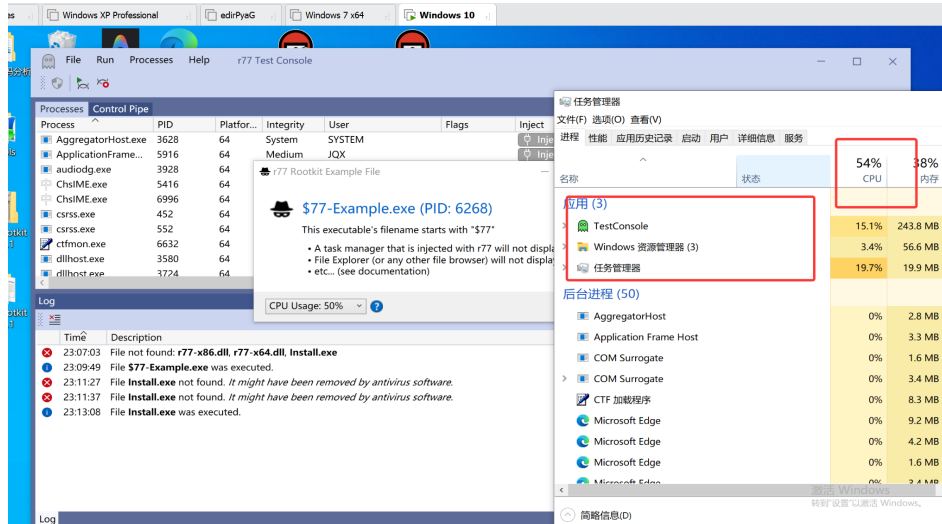


图 3: 隐藏生效后的任务管理器: 高 CPU 占用却无对应进程

(三) 文件系统隐藏验证

接下来验证文件隐藏功能。r77 默认配置隐藏所有以 \$77 为前缀的文件。

1. 实验步骤与现象

我在测试目录下尝试创建一个名为 \$77Secret.txt 的文件。

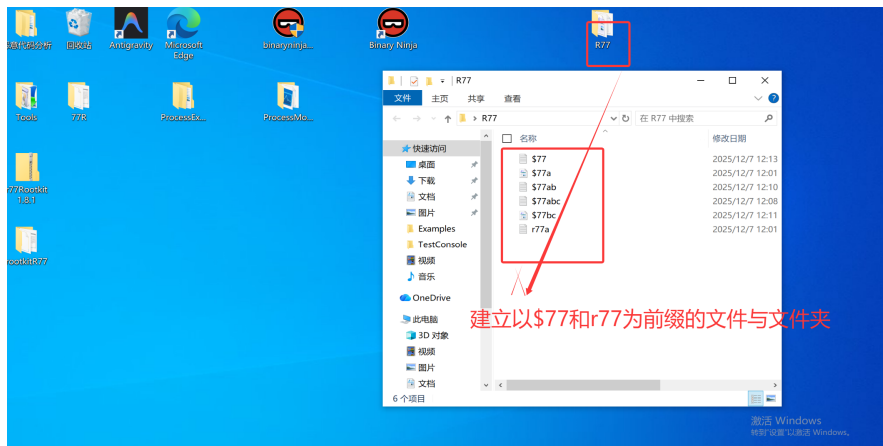
- **创建瞬间:** 鼠标右键 -> 新建 -> 文本文件。当我输入文件名并回车确认的瞬间, 文件图标凭空消失了。
- **验证存在性:** 我尝试再次在该目录下创建同名文件, 系统提示“文件已存在”。这证明文件确实被写入了磁盘 (NTFS 文件系统中有记录), 只是文件资源管理器 (Explorer.exe) “看不见”它。

原理分析: Explorer.exe 为了显示文件列表, 会调用 NtQueryDirectoryFile。r77 注入到 Explorer.exe 进程中的 DLL 截获了该调用, 遍历返回的文件信息数组, 一旦发现文件名匹配 \$77..., 就通过修改指针偏移量 (Offset) 的方式将该条目从数组中剔除。因此, 用户界面渲染出的列表中就不包含该文件。这种隐藏是实时的, 且对所有注入了 r77 的程序 (如 cmd.exe, PowerShell, Notepad 等) 都有效。

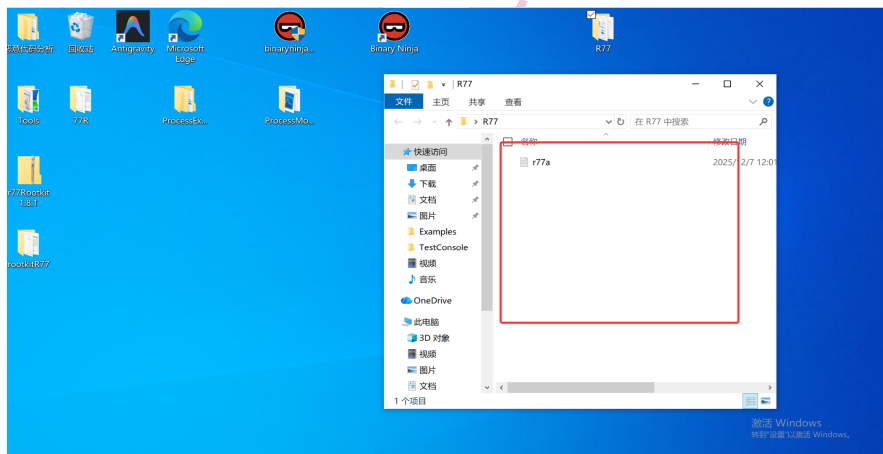
为了更清晰地展示这一过程, 我们将文件隐藏的前、后及卸载后的状态分别展示, 如图 4 所示。

(四) 注册表隐藏分析

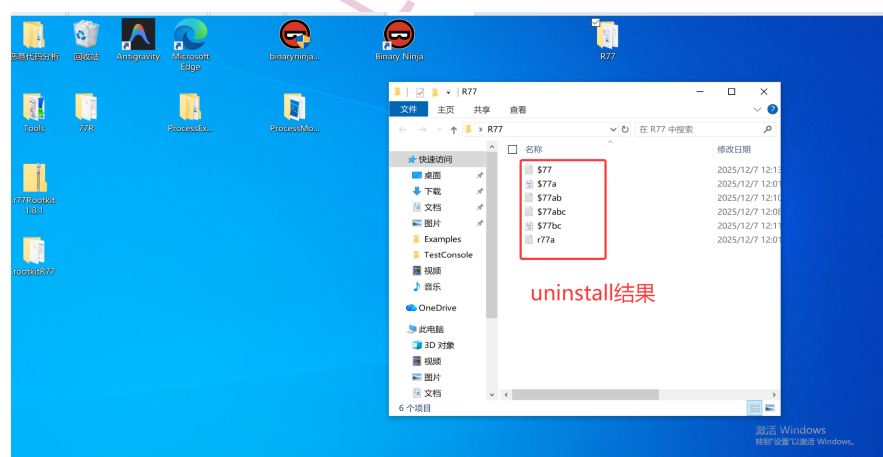
注册表隐藏是 r77 实现自我保护的关键一环。r77 的配置信息通常保存在 HKEY_LOCAL_MACHINE\SOFTWARE\\$77cor 等位置。若不对这些键值进行隐藏, 系统管理员可通过注册表编辑器 (RegEdit) 轻易发现异常



(a) 隐藏策略开启前：\$77 前缀文件正常显示



(b) 隐藏策略生效中：文件已创建但在视图中消失



(c) 卸载 r77 后：隐藏文件重新显现

图 4: 文件系统隐藏全周期验证过程（垂直视图以增加清晰度）

配置，进而追溯到 Rootkit 的存在。

1. 技术机制

注册表隐藏的实现原理与文件隐藏类似。当注册表编辑器展开某一键节点时，Windows 会调用 `NtEnumerateKey` 或 `NtEnumerateValueKey` 等 Native API 来枚举子键和键值。r77 通过 Hook 这些函数，在返回结果前对数据进行过滤：检查返回的键名（Key Name）或值名（Value Name），若检测到 \$77 前缀，则从返回缓冲区中删除该项，并调整后续条目的索引以保持连续性。经此处理后，管理员在注册表编辑器中将无法察觉 Rootkit 配置项的存在，显著增加了取证和清除的难度。

2. 实验验证

在目录 `HKEY_CURRENT_USER\Printers` 下添加注册表项 `HKEY_CURRENT_USER\Printers\$77-abc` 和 `HKEY_CURRENT_USER\Printers\r77-abc`，此时没有被隐藏，如下图所示。

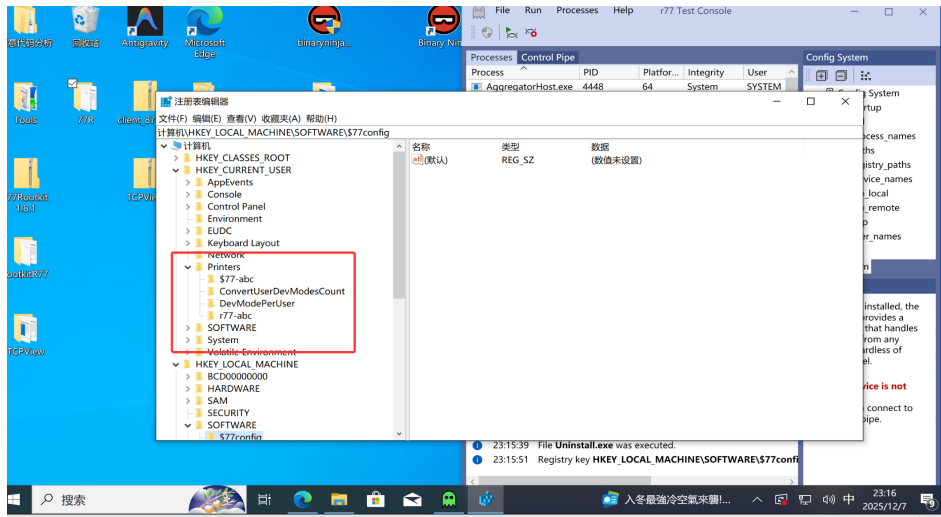


图 5: 注册表键隐藏前

执行 `install.exe`,\$77 为前缀的注册表项被隐藏，r77 为前缀的没有被隐藏：

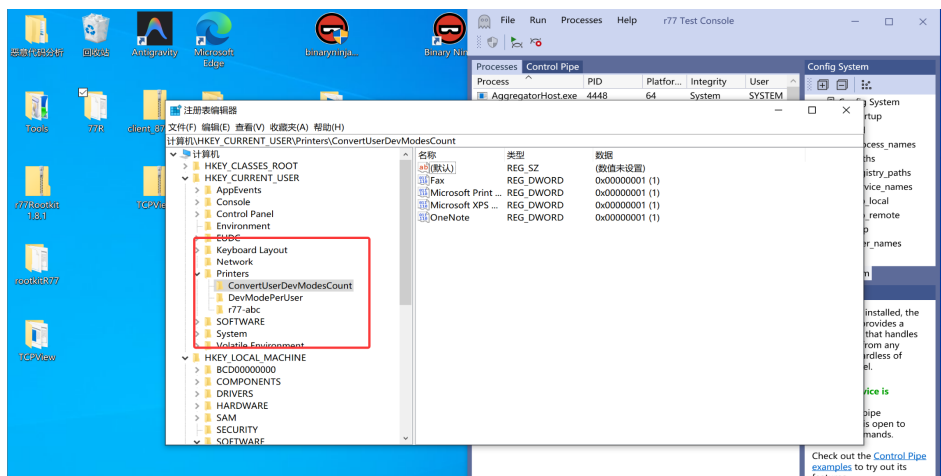


图 6: 注册表键被隐藏

3. 问题排查与解决

在本实验的注册表隐藏验证环节，我遇到了一个棘手的问题，排查过程耗费了相当的时间，但也加深了我对 Windows 注册表架构的理解。

问题现象：起初，我在 HKEY_CLASSES_ROOT\.*386 目录下创建了两个测试键：\$77-abc 和 r77-abc。然而，无论以普通用户权限还是管理员权限运行 Install.exe，这些注册表项始终未被隐藏。

排查尝试：

1. **配置文件检查：**我首先怀疑是 r77 的配置文件中，隐藏前缀并非默认的 \$77。于是尝试通过 TestConsole 界面编辑 Registry 配置项，但发现该界面为只读状态，无法修改。同时，在系统目录中也未找到对应的可编辑配置文件。
2. **Control Pipe 方式：**随后我尝试通过 r77 提供的 Control Pipe 机制动态添加隐藏规则，但 HKEY_CLASSES_ROOT\.*386 目录下的 \$77 前缀键仍然可见，隐藏功能未生效。

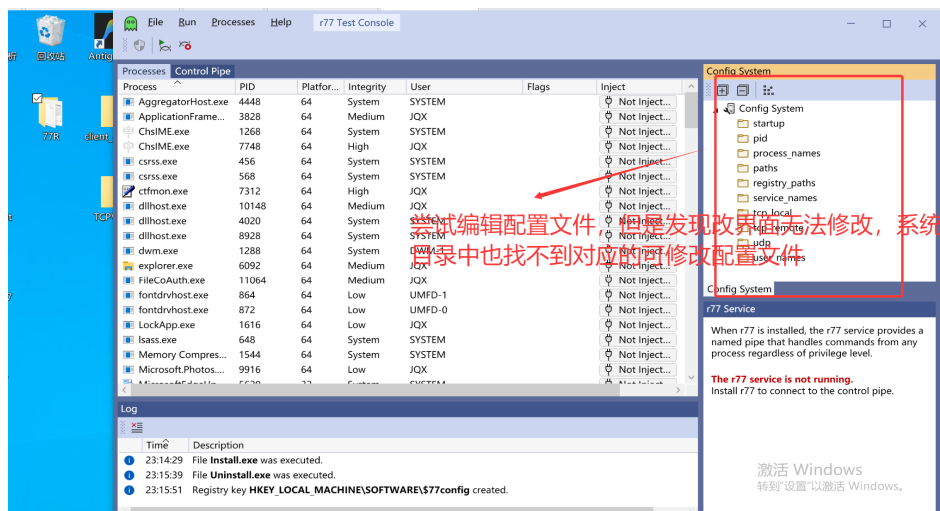


图 7: 排查过程：尝试通过 TestConsole 修改 Registry 配置项（界面为只读）

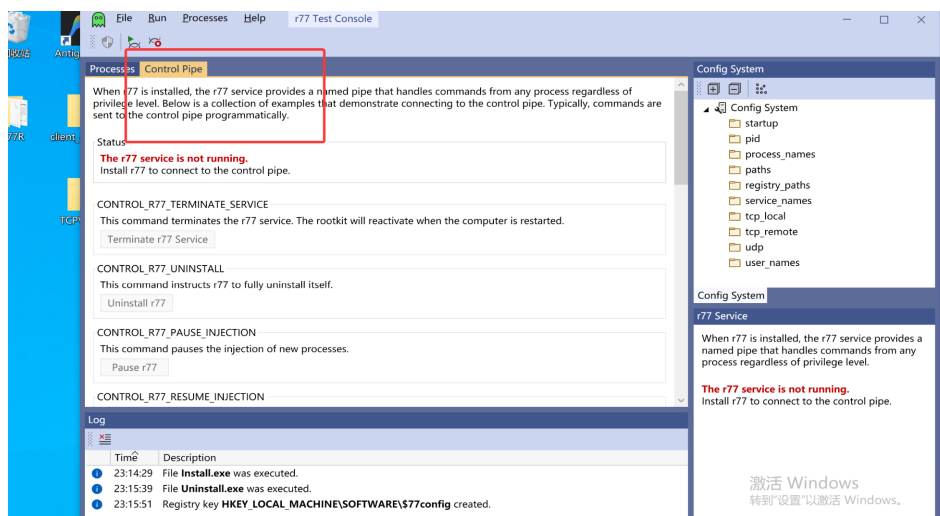


图 8: 排查过程：通过 Control Pipe 添加隐藏规则后，HKCR 下的键仍可见

问题解决:在多次尝试失败后,我决定更换测试目录。将注册表项创建于 HKEY_CURRENT_USER\Printers 路径下后,再次运行 Install.exe,注册表项终于被成功隐藏。

4. 根因分析

经过深入分析,我总结出导致上述现象的技术原因如下:

r77 的注册表隐藏功能依赖于对用户态进程的 API Hook,主要拦截 RegEnumKeyExW、RegOpenKeyExW、NtEnumerateKey 等函数。然而, HKEY_CLASSES_ROOT (HKCR) 并非一个独立的注册表 Hive,而是 HKEY_LOCAL_MACHINE\SOFTWARE\Classes 与 HKEY_CURRENT_USER\SOFTWARE\Classes 的联合视图 (Merged View)。

由于 HKCR 的这种特殊架构,其访问路径往往会绕过用户态的 API Hook,或由系统服务(如 COM 相关服务)代理完成访问。在这些场景下,r77 注入的 Hook 无法拦截到相关调用,自然也就无法实现隐藏。

相比之下, HKEY_CURRENT_USER (HKCU) 下的路径(如 Printers)是直接由用户态进程访问的标准 Hive。当注册表编辑器枚举这些路径时,必然调用 NtEnumerateKey 等 API, r77 的 Hook 能够完全生效,从而成功实现隐藏。

表 3: 不同注册表 Hive 的隐藏效果对比

注册表路径	隐藏效果	原因分析
HKEY_CLASSES_ROOT	失败	联合视图,访问路径可能绕过用户态 Hook
HKEY_CURRENT_USER	成功	标准 Hive, API 调用可被完全拦截
HKEY_LOCAL_MACHINE	部分成功	取决于具体子键的访问方式

(五) 网络连接隐藏验证

最后,我测试了对 TCP 连接的隐藏。这是对抗应急响应人员排查 C2 (Command & Control) 连接的重要手段。

1. 实验操作

1. 使用浏览器访问网站,产生一个随机高端口连接(实验中定位为端口 65172)。2. 使用 netstat -ano 确认该连接存在,并关联到浏览器进程 PID。3. 在 r77 配置中添加对端口 65172 的隐藏规则。

2. 结果与分析

如图 10a 和 10b 所示,隐藏后,无论是命令行的 netstat 还是图形化的 TCPView,都再也找不到该端口的踪迹。

技术细节:网络工具通常调用 GetExtendedTcpTable (iphlpapi.dll) 来获取系统的 TCP 连接表。r77 拦截该 API,解析返回的 MIB_TCPTABLE 结构。该结构包含一系列 MIB_TCPROW 行,每一行代表一个连接,包含本地 IP、端口、远程 IP、端口和所属进程 PID。r77 遍历这些行,比对端口号。一旦匹配,它通过内存拷贝 (Memory Copy) 将后续的所有行向前移动,覆盖掉当前行,并将总行数 (dwNumEntries) 减一。这种直接操作内存缓冲区的技术非常底层且高效,使得上层应用完全感知不到数据的缺失。

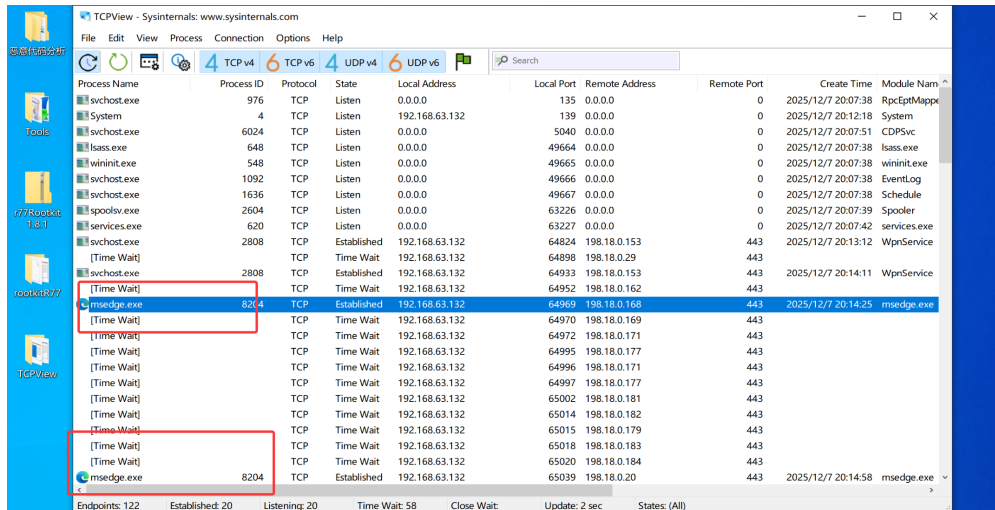
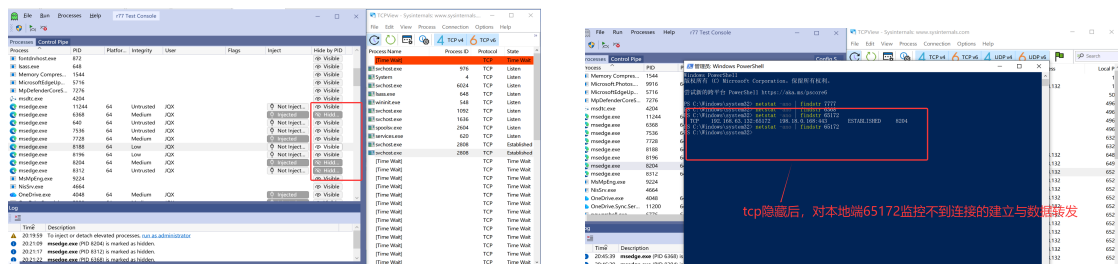


图 9: 隐藏前: Netstat 清晰显示端口 65172 的 ESTABLISHED 连接



(a) 隐藏后: Netstat 输出中该连接消失

(b) 隐藏后: TCPView 实时监控中亦无法捕获

图 10: 网络连接隐藏前后对比: 从应用层彻底抹除通信痕迹

四、 拓展阅读

（一） Detours 技术概述

Microsoft Detours 是微软研究院开发的一款用于在 Windows 上拦截二进制函数调用的库。虽然其初衷是用于调试、性能分析和热补丁 (Hot Patching)，但其强大的 Inline Hook 能力也常被 Rootkit 开发者利用。

在 r77 中，Detours 被用来修改内存中目标 API 的前几条指令，强行改变 CPU 的执行流 (PC 指针)，使其跳转到攻击者编写的 Hook Payload 中。

（二） Inline Hook 的实现细节

Detours 的核心操作是对函数入口的“外科手术式”修改。其标准流程如图 11 所示：

函数调用流程对比	
原始调用路径	⇒ User Code → Original API → Return
Hook 后调用路径	⇒ User Code → Trampoline (JMP) → Hook Function → Original API (Optional)

图 11: Detours 函数拦截流程模型

具体步骤如下：

1. **Trampoline 创建**：Detours 首先会申请一段内存 (Trampoline)，将目标函数 (Target Function) 开头被覆盖的指令 (通常是 5 字节以上) 复制到这里，并在末尾添加一条跳回原函数剩余部分的 JMP 指令。
2. **覆写入口 (Rewriting Target)**：目标函数的入口地址被修改为一条无条件跳转指令 (JMP rel32)，目标地址指向 Hook 函数。
3. **执行 Hook 逻辑**：当程序调用 API 时，首先跳转到 Hook 函数。在此处，r77 编写的代码会执行数据过滤 (如遍历链表删除名为 \$77 的节点)。
4. **回溯 (Optional)**：如果 Hook 函数需要原 API 的结果，它会调用 Trampoline，从而执行原函数的原始逻辑。

（三） r77 中的 Hook 策略图解

为了更直观地展示 r77 如何利用 Detours 截获数据，我绘制了如下的逻辑流程图 (图 12)。该图展示了调用方、系统 API 与恶意 Hook 函数之间的三角关系。

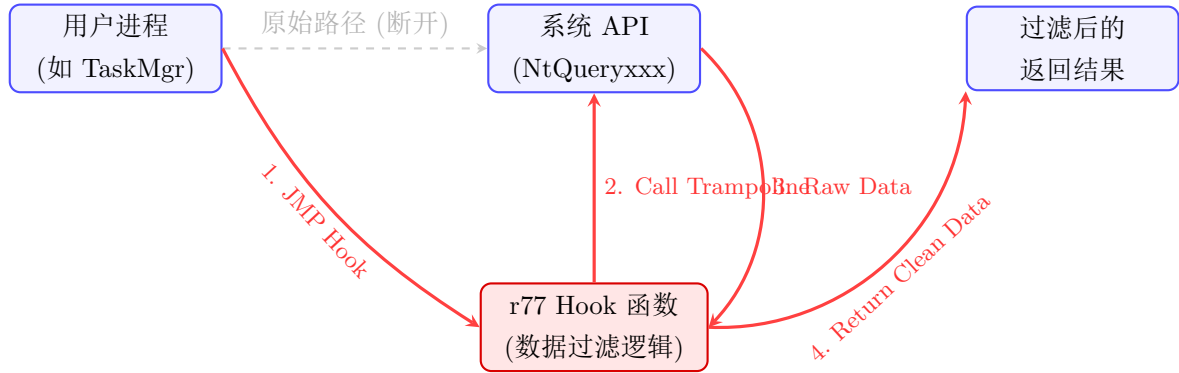


图 12: r77 Inline Hook 数据流向示意图

五、 总结

(一) 实验成果

本次实验围绕 r77 Rootkit 展开了全方位的分析与实践。通过在受控虚拟环境中部署这一用户态恶意软件，我系统地验证了其在进程、文件、注册表及网络层面的隐藏能力。实验结果表明，基于 API Hooking 的隐藏技术虽然不涉及 Windows 内核的修改，但对于依赖标准 Windows API 的监控工具（如任务管理器、资源管理器、Netstat）具有极强的欺骗性。

在技术原理层面，通过深入研究 Microsoft Detours 机制，我理解了 Inline Hook 是如何通过修改函数入口指令、构造 Trampoline 跳板来实现执行流劫持的。这不仅解释了 r77 的工作方式，也为后续研究用户态 Rootkit 的检测方法提供了理论基础。

(二) 关键发现

在实验过程中，我获得了以下几点重要发现：

- CPU 负载异常作为检测特征：**当 r77 隐藏高 CPU 占用进程后，任务管理器显示的“各进程 CPU 占用之和”将明显小于“系统总 CPU 占用率”。这种数据不一致现象可作为检测用户态 Rootkit 的重要线索。
- 注册表 Hive 架构对 Hook 有效性的影响：**通过对 HKEY_CLASSES_ROOT 与 HKEY_CURRENT_USER 的对比测试，我发现 r77 的注册表隐藏功能仅对标准 Hive 有效，而对联合视图（Merged View）类型的虚拟 Hive 无效。这一发现揭示了用户态 Rootkit 的固有局限性——其隐藏能力高度依赖于目标 API 的调用路径是否经过被 Hook 的用户态函数。
- 反射式 DLL 注入的隐蔽性：**r77 采用的 Reflective DLL Injection 技术使其 Payload DLL 不出现在进程的模块列表（PEB.Ldr）中，传统的模块枚举检测方法对其无效。

(三) 检测思路

基于本次实验的分析，我总结了以下几种针对用户态 Rootkit 的检测思路：

表 4: 用户态 Rootkit 检测方法汇总

检测方法	原理说明
直接系统调用 (Direct Syscall)	绕过 ntdll.dll 中被 Hook 的函数，直接通过 <code>syscall</code> 指令进入内核，获取未被篡改的原始数据
代码完整性校验	比对内存中 API 函数入口的指令与磁盘上原始 DLL 的内容，检测是否存在异常的 JMP 指令
内核级监控	使用 ETW (Event Tracing for Windows) 或内核驱动获取进程/网络信息，绕过用户态 Hook
内存取证分析	扫描进程内存中的可疑代码段，检测反射式注入的 DLL 残留

(四) 实验心得

总体而言，本实验达成了既定目标，显著加深了我对 Windows 恶意代码持久化与隐藏技术的理解。在注册表隐藏验证环节遇到的问题及其解决过程，也让我对 Windows 注册表的分层架构有了更清晰的认识——`HKEY_CLASSES_ROOT` 作为 `HKLM` 与 `HKCU` 的联合视图这一设计，在提供便利性的同时，也为安全研究带来了复杂性。

通过本次实践，我不仅掌握了 Rootkit 的隐藏原理，也积累了分析和对抗此类恶意代码的实战经验，为后续深入研究基于内存取证的检测方法奠定了基础。